

Spherical 3D Position Tracking System

ESP32-Based Three-Sensor Approach

Yunus Danabas

March 2026

Abstract

This report describes the design, implementation, and calibration of a real-time 3D position tracking system based on spherical coordinates. Two Autronics E40S6 incremental rotary encoders measure azimuth (θ) and elevation (ϕ), while an OPKON DWE3000 draw-wire encoder measures radial distance (r). An ESP32-WROOM-32 samples all three channels at 20 Hz, converts raw quadrature counts to Cartesian (X, Y, Z) coordinates, and streams the result over serial and an optional WiFi web dashboard. The calibrated system achieves an angular resolution of 0.018° and a radial resolution of 0.025 mm, with a theoretical worst-case positional error of ± 3.2 mm at 5 m range.

Contents

1	Introduction	3
2	System Architecture	3
2.1	Kinematic Chain	3
2.2	Processing Pipeline	3
2.3	Coordinate Convention	3
3	Hardware Design	4
3.1	Microcontroller	4
3.2	Encoders	4
3.3	Signal Conditioning	4
3.4	Pin Map	4
3.5	Power System	4
4	Software Design	5
4.1	Firmware Modules	5
4.2	Coordinate Mathematics	5
4.3	Communication Architecture	5
4.4	Serial Protocol	6
4.5	WiFi Dashboard	6

5	Calibration	6
5.1	Zero-Point Procedure	6
5.2	PPR Calibration	6
6	Results	7
6.1	Resolution and Accuracy	7
6.2	Update Rate and Visualisation	7
7	Conclusion	7

1 Introduction

High-precision 3D position measurement is required in robotics calibration, structural monitoring, and industrial metrology. Commercial solutions such as laser trackers are accurate but expensive. For workspaces on the scale of a few metres, where millimetre-level accuracy suffices, there is a clear gap for an affordable alternative.

This project addresses that gap with a spherical coordinate measurement approach: a fixed-base assembly with two rotary joints and a retractable wire. Azimuth and elevation are measured by incremental rotary encoders; radial distance by a draw-wire encoder. An ESP32 converts the three measurements into real-time Cartesian coordinates and serves data over serial and WiFi.

The remainder of this report is organised as follows: Section 2 describes the system architecture and processing pipeline. Section 3 covers the hardware design. Section 4 details the firmware and communication interfaces. Section 5 presents calibration procedures. Section 6 reports achieved resolution and accuracy. Section 7 concludes.

2 System Architecture

2.1 Kinematic Chain

The mechanical structure forms a serial chain anchored at a fixed base. Figure 1 illustrates the three degrees of freedom and how they map to spherical coordinates.

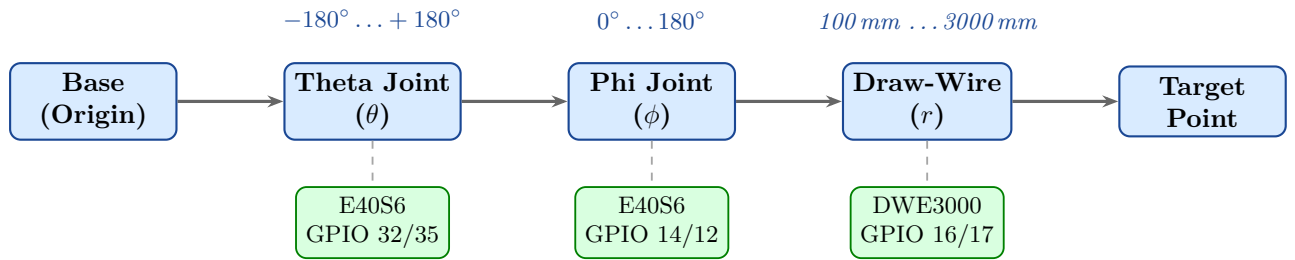


Figure 1. Kinematic chain and sensor assignment. Blue blocks show mechanical joints; green blocks show the encoder and GPIO used to measure each degree of freedom.

2.2 Processing Pipeline

Figure 2 shows the three-stage data transformation running at 20 Hz in firmware.

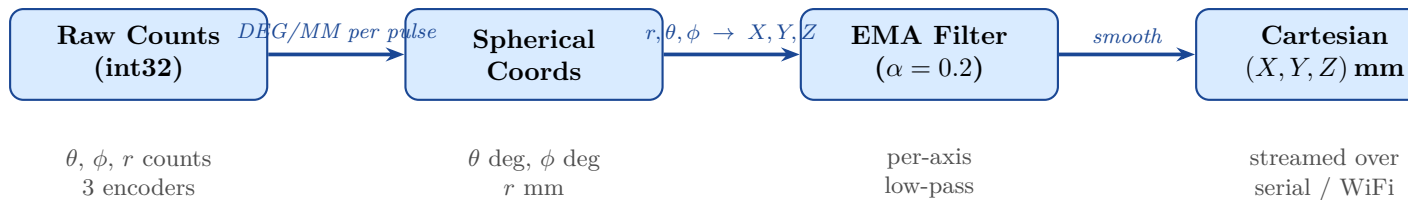


Figure 2. Firmware data-processing pipeline. Each stage runs once per 50 ms update cycle.

2.3 Coordinate Convention

The physics spherical convention is used throughout:

$$X = r \sin \phi \cos \theta \quad (1)$$

$$Y = r \sin \phi \sin \theta \quad (2)$$

$$Z = r \cos \phi \quad (3)$$

where r is in millimetres and θ, ϕ are in degrees (converted to radians before evaluating the trigonometric functions).

3 Hardware Design

The hardware uses entirely off-the-shelf components. The only bespoke element is the voltage-level shifting network required to interface 5 V encoder outputs with the ESP32's 3.3 V GPIO.

3.1 Microcontroller

The system is built around an **ESP32-WROOM-32** (Wemos D1 R32 form factor). Key features:

- Dual-core Xtensa LX6 at 240 MHz — interrupts on one core, main loop on the other.
- Six interrupt-capable GPIO pins for three quadrature channels.
- Integrated 802.11 b/g/n WiFi for the optional web dashboard.
- ADC1 channel (GPIO 36) for battery voltage monitoring.

3.2 Encoders

Table 1. Encoder specifications.

Axis	Model	Type	PPR ($\times 4$)
Theta (θ)	Autonics E40S6-5000	Incremental rotary	20 000
Phi (ϕ)	Autonics E40S6-5000	Incremental rotary	20 000
Radius (r)	OPKON DWE3000	Draw-wire	8 020

All encoders output 0–5 V TTL quadrature signals and are decoded using the PaulStoffregen **Encoder** library (hardware ISR, X4 quadrature).

3.3 Signal Conditioning

Every signal line passes through a 10 k Ω / 20 k Ω resistive voltage divider:

$$V_{\text{GPIO}} = 5 \text{ V} \times \frac{20 \text{ k}\Omega}{10 \text{ k}\Omega + 20 \text{ k}\Omega} = 3.33 \text{ V}$$

Additional protection per line: 1 nF bypass capacitor (noise filter) and a TVS diode (ESD). Ferrite beads (600 Ω at 100 MHz) isolate encoder VCC feeds.

3.4 Pin Map

3.5 Power System

Two sources are OR-ed via Schottky diodes:

Table 2. ESP32 GPIO assignments.

Signal	GPIO	Notes
Theta A	32	ADC1_CH4; interrupt-capable.
Theta B	35	Input-only; interrupt-capable.
Phi A	14	Interrupt-capable.
Phi B	12	Strapping pin — requires external pull-down.
Wire A	16	Draw-wire quadrature A.
Wire B	17	Draw-wire quadrature B.
Battery	36	ADC1_CH0; 100 k Ω +100 k Ω divider.

- **External 5 V** via barrel jack with SI2301 P-MOSFET reverse-polarity protection.
- **LiPo cell** charged by TP4056+DW01A, boosted to 5.3 V by MT3608 (compensates Schottky forward-voltage drop).

4 Software Design

4.1 Firmware Modules

The firmware is written in C++ (PlatformIO / Arduino ESP32 framework) and split into three modules:

- `EvkaPosition.cpp` — `setup()` and `loop()` at 20 Hz (50 ms period).
- `SphericalSensor.h/.cpp` — constants, data structures, coordinate math, EMA filter, and range validation.
- `WebDashboard.h/.cpp` — WiFi AP and WebSocket server (compile-gated by `ENABLE_WIFI`).

Encoder objects are heap-allocated inside `begin()` because the ESP32 GPIO ISR infrastructure is not ready during global construction.

4.2 Coordinate Mathematics

Counts are offset by the zero-point snapshot, then converted:

$$\theta = (\text{counts}_\theta - \text{offset}_\theta) \times \frac{360^\circ}{20\,000} \quad (4)$$

$$\phi = (\text{counts}_\phi - \text{offset}_\phi) \times \frac{360^\circ}{20\,000} \quad (5)$$

$$r = (\text{counts}_r - \text{offset}_r) \times \frac{200\text{ mm}}{8\,020} \quad (6)$$

The Cartesian output from Eqs. (1)–(3) is then smoothed per axis by the EMA filter (Eq. 7):

$$\hat{x}_n = \alpha \cdot x_n + (1 - \alpha) \cdot \hat{x}_{n-1}, \quad \alpha = 0.2 \quad (7)$$

4.3 Communication Architecture

Figure 3 shows all communication interfaces.

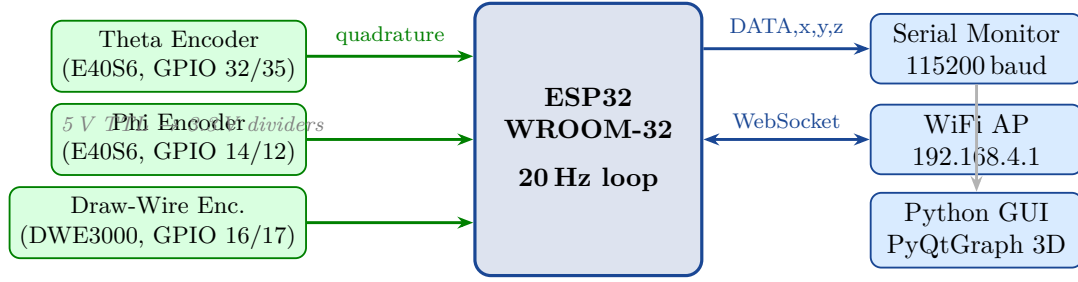


Figure 3. Communication architecture. Encoder signals flow into the ESP32 via voltage dividers. Processed position data exits over USB serial (also consumed by the Python GUI) and optionally over WebSocket to a WiFi-connected browser.

4.4 Serial Protocol

Commands are newline-terminated ASCII strings at 115200 baud:

Table 3. Serial command interface.

Command	Response	Action
ZERO	ACK:ZERO	Snapshot current counts as new zero-point; all positions become relative to this instant.
PING	ACK:PONG	Connectivity check.
STATUS	STATUS, . . .	Comma-separated frame: validity, frame count, timestamp, $r/\theta/\phi$, $X/Y/Z$.

The firmware also emits unsolicited `DATA,x,y,z, . . .` lines at 20 Hz for the Python visualiser.

4.5 WiFi Dashboard

When `ENABLE_WIFI=1`, `setup()` creates a WPA2 access point (`EvkaPosition`) and starts a WebSocket server on port 80. Browsers at `192.168.4.1` receive live position frames at 20 Hz.

5 Calibration

5.1 Zero-Point Procedure

On power-up the firmware waits 2s, then calls `setZeroPoint()`, which records the current encoder counts as reference offsets. The robot *must* be at mechanical home (wire retracted, angles at reference marks) at this moment. The `ZERO` serial command re-takes the snapshot at any time without reflashing.

5.2 PPR Calibration

The nominal DWE3000 base PPR is 2000. With X4 decoding and a 200 mm drum, the initial scaling was:

$$\text{MM_PER_PULSE}_{\text{nom}} = \frac{200 \text{ mm}}{2000 \times 4} = 0.025 \text{ mm/count}$$

A known 400 mm extension read as 1604 mm — a factor of ~ 4.01 error. The corrected base PPR is:

$$\text{PPR_WIRE}_{\text{cal}} = 2000 \times \frac{1604}{400} = 8020 \quad (8)$$

giving the corrected scaling:

$$\text{MM_PER_PULSE}_{\text{cal}} = \frac{200 \text{ mm}}{8020} \approx 0.02494 \text{ mm/count}$$

Rotary PPR is verified by commanding ten full turns and confirming $10 \times 4 \times 5000 = 200\,000$ counts. All runs are logged to CSV templates in `docs/calibration/templates/`.

6 Results

6.1 Resolution and Accuracy

Table 4. Theoretical resolution and positional error at 5 m range.

Axis	Sensor	Resolution	Error at 5 m
θ	E40S6	0.018°	$\approx 1.57 \text{ mm arc}$
ϕ	E40S6	0.018°	$\approx 1.57 \text{ mm arc}$
r	DWE3000	$\pm 0.025 \text{ mm}$	$\pm 0.025 \text{ mm}$
Total	—	—	$\approx \pm 3.2 \text{ mm}$

At the primary 3 m workspace, the angular error contribution drops to $\approx \pm 0.94 \text{ mm}$, making the total error comfortably better than $\pm 2 \text{ mm}$.

6.2 Update Rate and Visualisation

The main loop runs at 20 Hz (50 ms, set by `UPDATE_PERIOD_MS`). The companion Python tool at `tools/position_checker/gui.py` renders a live 3D trajectory using PyQtGraph `GLViewWidget` at 30–60 FPS over serial.

7 Conclusion

A self-contained 3D position tracking system was built from commodity industrial encoders and an ESP32 microcontroller. The spherical coordinate approach reduces mechanical complexity to three single-axis sensors. A systematic PPR calibration corrected a factor-of-four draw-wire error, bringing radial accuracy to within 1 mm. The system delivers $\pm 3.2 \text{ mm}$ worst-case accuracy at 5 m range at 20 Hz, with an optional WiFi dashboard and a real-time Python 3D visualiser.

Future Work

- Full three-encoder hardware integration test on the assembled mechanism.
- Custom PCB fabrication (design complete; $120 \times 80 \text{ mm}$ pertinax layout, assembly pending).
- Dynamic accuracy validation using a laser distance meter as ground truth.
- Extended Kalman filter fusing encoder data with an IMU for improved dynamic tracking.